Hardware/Software Co-Design of a Camera-Based Driver Assistance System

Real-Time Object Detection on an Embedded Platform

Author: Moshiour Rahman Prince
Course: HW/SW Codesign
Supervisor: Prof. Dr. Achim Rettberg
Institute: University of Applied Sciences Hamm-Lippstadt
Date: May 11, 2026

Abstract — This report documents the hardware/software co-design of a camera-based Advanced Driver Assistance System (ADAS) capable of real-time object detection. The embedded systems (ES) design methodology covered in the lecture is applied throughout: the system is decomposed into hardware and software processing elements (PEs), an architecture model is constructed from those PEs, end-to-end latency is estimated stage by stage, and functional safety obligations under ISO 26262 are addressed. The central design principle holds that behaviour is defined independently of any implementation medium and is only mapped to hardware or software after systematic exploration of all partitioning criteria.

1. System Partitioning

Architecture exploration is the opening step of the ES design methodology: processing elements are instantiated and each behaviour in the specification is assigned to exactly one PE [1]. Three evaluation criteria drive every assignment decision in the ADAS context.

1.1 Partitioning Criteria

Deterministic timing. Hard real-time deadlines govern safety-critical perception tasks. Dedicated hardware accelerators operate in a cycle-accurate, jitter-free manner, whereas software executing on a general-purpose core is subject to interrupt latency and OS scheduling variance. Consequently, every task that lies on the critical timing path is assigned to custom hardware.

Power efficiency. Automotive-grade embedded platforms operate under tight thermal and electrical constraints. A fixed-function hardware block (FPGA logic, ASIC accelerator) accomplishes a given computation with far fewer energy-per-operation than an equivalent software routine on a programmable core.

Adaptability. Classification thresholds, neural network model weights, and communication stack parameters are revised across vehicle derivatives and over-the-air (OTA) update cycles. Software accommodates such changes without requiring hardware re-fabrication, making it the natural host for any functionality that must remain configurable.

1.2 Hardware Processing Element (PE 2)

- **Image Signal Processor (ISP).** Handles MIPI CSI-2 frame capture, demosaicing, and noise reduction in a continuous pixel-stream pipeline. Hardware is required because the data rate and strict frame-synchronisation timing cannot be met in software.
- **FPGA Preprocessing Accelerator.** Performs colour-space conversion, spatial resizing, and per-channel normalisation in a fully parallel pipeline with bounded, deterministic latency.
- **Neural Processing Unit / GPU.** Executes CNN inference for object detection. Its massively parallel array delivers the required throughput at a power envelope orders of magnitude below a CPU baseline.
- **AXI4 Bus Interface FSM.** A synthesised finite-state machine enforces the AXI4 handshake protocol, guaranteeing cycle-accurate bus transactions between PE 2 and PE 1 [1].

1.3 Software Processing Element (PE 1)

- **Real-Time Operating System (RTOS).** Provides task scheduling (static or dynamic) and interrupt management without touching hardware.
- **Decision and Control Logic.** Evaluates bounding-box detections, applies configurable confidence thresholds, and dispatches actuator commands.
- **CAN / Ethernet Stack.** Implements higher-layer AUTOSAR COM framing; kept in software because protocol versions differ across vehicle programmes.
- **Diagnostics and OTA Management.** Handles OBD-II fault reporting and orchestrates firmware update delivery on the application processor.

Table 1 summarises the full partitioning, scoring each behaviour against the three criteria.

Table 1. HW/SW Partitioning Matrix (● = primary driver, ○ = not primary driver).

Behaviour	PE	Timing	Power	Flex.
ISP / frame capture	HW (PE 2)	●	●	○
FPGA preprocessing	HW (PE 2)	●	●	○
CNN inference (NPU)	HW (PE 2)	●	●	○
RTOS / scheduling	SW (PE 1)	○	○	●
Decision / control	SW (PE 1)	○	○	●
CAN / ETH stack	SW (PE 1)	○	○	●
Diagnostics / logging	SW (PE 1)	○	○	●

2. Architecture Design

Once all behaviours are assigned, the specification model is promoted to an *architecture model* [1]: two PEs are connected by a shared bus and each subsystem is refined to the component level.

2.1 System Block Diagram

Figure 1 presents a hierarchical block diagram of the ADAS. The diagram is organised in four tiers: the top-level system block, three major subsystems, the internal decomposition of the Processing Unit, and the communication interfaces. Arrow styles distinguish data flow, composition, and bidirectional exchange.

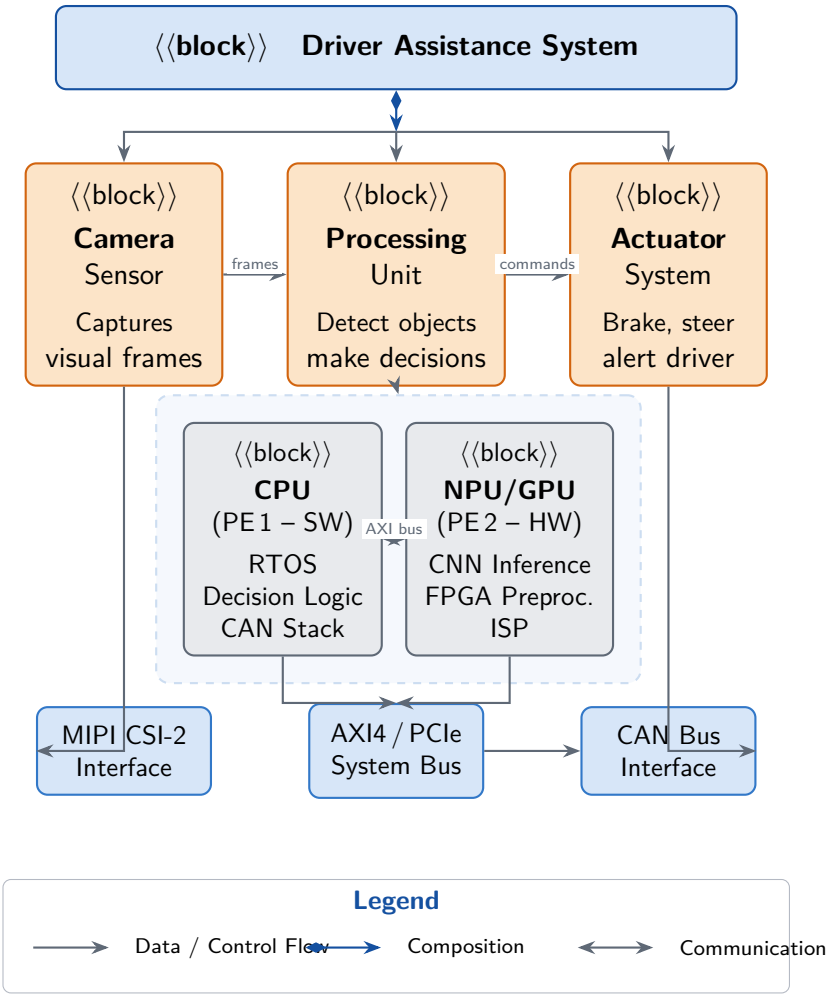

Figure 1. ADAS block diagram (four tiers). Orange: top-level subsystems. Blue: system block and buses. Gray: internal processing components.

2.2 Data Flow Through the System

The processing chain follows the synthesis flow described in the lecture (slide 4):

1. Raw video frames are transferred from the camera over **MIPI CSI-2** into the ISP block on PE 2.
2. The FPGA accelerator on PE 2 resizes, colour-converts, and normalises each frame, writing the resulting tensor into shared memory.
3. The GPU/NPU reads the tensor, runs CNN inference, and writes bounding-box predictions back to memory.
4. Detection results are forwarded to PE 1 across the **AXI4 System Bus** using the double-handshake protocol (address / data / ready / ack) described in the lecture communication model [1].

5. The RTOS on PE 1 wakes on interrupt; the decision-logic software evaluates detections and issues actuator commands via the **CAN bus** to the braking, steering, and warning ECUs.

2.3 Communication Interfaces

- **MIPI CSI-2.** High-speed differential serial link connecting the camera module to the ISP; supports frame rates up to 120 fps at full HD resolution.
- **AXI4 / PCIe System Bus.** Connects PE 2 (bus master) to PE 1 (bus slave). A synthesised double-handshake FSM guarantees protocol correctness and bounded transfer latency (slides 19–24).
- **CAN Bus.** Connects the ADAS ECU to the braking, steering, throttle, and warning actuator nodes on the in-vehicle network.

3. Performance Analysis

3.1 End-to-End Latency Requirement

At a cruising speed of 50 km/h a vehicle advances roughly 14 m per second. To permit a comfortable braking intervention, the full perception-to-actuation chain must conclude well within 100 ms. A system-level target of **50 ms** is adopted here, providing a 2× margin over the absolute physical limit.

3.2 Stage-by-Stage Latency Budget

Each leaf behaviour in the specification model is annotated with a `waitfor(delay)` timing estimate following the approach introduced in slide 15 of the lecture. Table 2 lists these estimates together with cumulative totals.

Table 2. End-to-end latency budget per processing stage.

Stage	Delay	Cumulative
Camera exposure & readout	10 ms	10 ms
ISP: demosaicing + CSI-2	3 ms	13 ms
FPGA preprocessing	5 ms	18 ms
GPU/NPU CNN inference	20 ms	38 ms
AXI bus DMA transfer	2 ms	40 ms
SW decision logic (RTOS)	5 ms	45 ms
CAN actuator command	2 ms	47 ms
Total		**47 ms**

The 47 ms aggregate satisfies the 50 ms requirement and leaves a 3 ms contingency for measurement uncertainty and bus arbitration.

3.3 Bottleneck Analysis

CNN inference (20 ms, 43 % of budget). This single stage dominates the timing budget. A pure CPU implementation of the same network would require 200–400 ms, violating the deadline by a factor of four to eight. Offloading inference to the GPU/NPU reduces it to 20 ms by exploiting massive data-level parallelism.

AXI bus transfer (2 ms). At current detection output sizes this contribution is modest. However, if the pipeline is scaled to higher-resolution feature maps, bus bandwidth could become a secondary bottleneck.

Camera readout (10 ms, fixed by frame rate). A 30 fps sensor imposes a fixed 10 ms frame-period floor. Upgrading to a 60 fps sensor would halve this contribution.

3.4 Optimisation Strategies

- **INT8 quantisation on the NPU.** Replacing 32-bit floating-point with 8-bit integer arithmetic reduces inference time from 20 ms to approximately 10 ms with negligible accuracy loss.
- **FPGA frame pipelining.** Overlapping preprocessing of frame n with NPU inference on frame $n-1$ effectively hides the 5 ms preprocessing cost behind the longer inference stage.
- **DMA transfers on the AXI bus.** Direct memory access removes CPU involvement from data movement, keeping bus occupancy and transfer latency at their minimum.
- **Static scheduling on PE 1.** A fixed, pre-computed task execution order (slide 16) eliminates OS scheduling jitter and guarantees the 5 ms WCET of the decision-logic task.

4. Verification and Safety

4.1 Verification Across Abstraction Levels

The ES design flow mandates a parallel *validation track* at every synthesis step (slide 4): each model is compiled into a simulation executable, subjected to validation and estimation, before the next refinement proceeds.

Functional (untimed) simulation. The specification model is executed in SpecC or SystemC to confirm that the detection algorithm produces correct outputs across a representative stimulus set before any timing constraints are imposed.

Architecture-level co-simulation. Once the system is partitioned, PE 1 runs as an Instruction-Set Simulator (ISS) and PE 2 behaviours are represented as cycle-accurate RTL models. This allows timing violations to be found at the model level, before silicon is committed [1].

Hardware-in-the-Loop (HIL) testing. The physical ADAS ECU is placed in a closed-loop test rig. Pre-recorded camera and radar scenarios are injected while real actuator feedback is monitored. HIL reveals bus-arbitration conflicts and interrupt-handling issues that simulation abstracts away.

Formal model checking. Safety-critical FSMs — chiefly the AXI bus interface FSM (slide 42) — are verified with a model checker to prove absence of deadlock and ensure full compliance with the AXI4 protocol specification.

4.2 Functional Safety under ISO 26262

Automotive functional safety is governed by **ISO 26262** [2] (*Road Vehicles — Functional Safety*), which assigns each function an Automotive Safety Integrity Level (**ASIL A–D**). A camera-based automatic emergency braking function typically

warrants **ASIL B** or **ASIL C**, depending on the severity and exposure analysis in the HARA.

The following ISO 26262 requirements are relevant to this architecture:

- **Sensor redundancy.** A secondary modality (radar or a second camera) must independently confirm a detection before the braking ECU is authorised to act.
- **Diagnostic coverage.** The NPU and FPGA incorporate Built-In Self-Test (BIST) logic and error-detection codes (EDC) to satisfy the ASIL-assigned hardware diagnostic coverage target.
- **Safe-state transition.** Any detected fault triggers a transition to a defined safe state — suppression of autonomous braking and issuance of a driver alert — within the fault-tolerant time interval.
- **Freedom from interference.** The ARM Cortex-A on PE 1 uses a Memory Protection Unit (MPU) to isolate safety-relevant software partitions from diagnostic and OTA management tasks.

4.3 Runtime Fault Scenarios

1. **Camera signal loss.** A frame-synchronisation timeout in the ISP triggers an interrupt on PE 1. The RTOS activates degraded mode: autonomous braking is inhibited, a visual and audible driver alert is raised, and the fault is logged over CAN.
2. **NPU inference timeout.** A hardware watchdog monitors the AXI-master channel. On timeout the watchdog asserts a reset to the NPU, the bus interface FSM re-forwards the most recent valid detection for one frame, and a transient-fault flag is set in the diagnostic log.
3. **AXI bus protocol error.** If the double-handshake FSM does not receive an `ack` within the window `range(t3; t4; 10; 25)` (slide 42), it re-enters the request state. After three consecutive failures a full bus reset sequence is initiated.
4. **RTOS decision-task overrun.** If the decision-logic task misses its WCET deadline, the RTOS safety handler vetoes any pending actuator command and generates a driver-takeover request via the HMI display.

5. Conclusion

This report demonstrated the application of the ES HW/SW co-design methodology to a real-time camera-based ADAS. The architecture partitions the system into a hardware PE comprising an ISP, FPGA preprocessing accelerator, and GPU/NPU, and a software PE running an RTOS on an ARM Cortex-A processor. The two PEs communicate over an AXI4 bus governed by a formally synthesised double-handshake protocol.

The four-tier SysML block diagram documents the architecture from the top-level system block down to individual actuator interfaces. The end-to-end latency estimate of **47 ms** satisfies the 50 ms deadline, and the principal bottleneck — GPU/NPU inference — is mitigated by INT8 quantisation and FPGA frame pipelining. Safety compliance at **ASIL B/C** is achieved through sensor fusion redundancy, BIST-based diagnostic coverage, MPU memory partitioning, and four explicitly defined runtime fault-handling procedures.

References

[1] A. Rettberg, *HW/SW Codesign — Lecture Slides*, University of Applied Sciences Hamm-Lippstadt, Apr. 2025.

[2] ISO, *ISO 26262: Road Vehicles — Functional Safety*, 2nd ed., International Organisation for Standardisation, 2018.

[3] OMG, *OMG SysML Specification, v1.6*, Object Management Group, Nov. 2019.

[4] ARM Ltd., *AMBA AXI and ACE Protocol Specification*, ARM IHI0022H, 2021.